Windowed Alerting and a Composite Integrity Index for Two-Span Bridge Structural Health Monitoring on a Serverless AWS Pipeline

Kasireddy Vadicherla
Student ID: X25104047
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25104047@student.ncirl.ie

***Abstract*—Bridge decks accumulate fatigue and thermal-cycle damage between the periodic visual inspections that most authorities still rely on, leaving a gap that continuous instrumentation is well placed to close. This report documents a two-span bridge monitoring system built around five simulated structural sensors per span (strain, deck vibration, tilt, traffic load, and expansion-joint movement) feeding a Bottle-based fog node that windows, aggregates, and threshold-evaluates every reading before dispatching a compact summary onward. Aggregates are batched into Amazon SQS, consumed by an AWS Lambda ingestion function into DynamoDB, and served to a dashboard by a second Lambda answering behind a real API Gateway REST endpoint through a single Python structural-pattern-matching dispatch statement. A composite structural integrity index folds each window's strain average and vibration peak into a single 0–100 score whose critical bounds are set equal to the fog node's own alert thresholds, so the score reaches zero exactly where an alert would already be firing. A 115-test automated suite, including a real-socket HTTP test file that drives the exact WSGI server class used in production over an actual TCP connection, passes in full. The system is deployed to a real AWS Academy Learner Lab account (661886400169) via a single Terraform apply of 24 resources, with a pre-deployment code audit finding and correcting a DynamoDB scan-pagination undercount and unbatched SQS publishing before either reached the live account. Post-deployment verification against the live endpoint confirmed all four pipeline health fields true, one hundred items landed in DynamoDB within the observation window, and a climbing structural-integrity-index sequence for both spans driven by real generated sensor data.**

***Keywords*—structural health monitoring, fog computing, Internet of Things, serverless computing, Amazon SQS, Amazon DynamoDB, AWS Lambda, bridge instrumentation**

I. Introduction

Visual inspection remains the primary means by which most road authorities assess bridge condition, typically on a cycle measured in years rather than days. Between inspections, a deck accumulates strain cycles from traffic loading, thermal expansion and contraction at its joints, and slow deformation that a scheduled walk-through has no way of catching as it develops. The case for closing that gap with instrumentation rather than waiting for the next inspection window is not new: Farrar and Worden's foundational survey frames structural health monitoring as exactly this shift, from periodic manual assessment toward a damage-identification process built on data gathered continuously from a structure in service [1]. Bridges specifically have been a proving ground for that shift for two decades: the Golden Gate Bridge wireless sensor deployment demonstrated that hundreds of strain and vibration nodes could report reliably over a structure's own length without a wired backbone [2], and a broader survey of large-scale bridge monitoring installations across multiple countries catalogues strain gauges, accelerometers, and inclinometers as the recurring instrumentation choices once a bridge authority commits to continuous sensing [3]. The sensor set built for this project (strain, deck vibration, tilt, traffic load, and expansion-joint movement) follows that same established combination rather than an arbitrary one.

Instrumenting a structure is only half the problem; the other half is what happens to the resulting stream of readings before an engineer ever sees them. Streaming every raw sample from ten sensor sources to a distant data centre wastes bandwidth on readings that, individually, rarely matter, and it delays exactly the moment a threshold breach needs to be flagged. Fog computing exists specifically to intercept that stream before it reaches the distant data centre: a compute tier co-located with the sensors themselves can turn ten seconds of raw samples into one short summary long before bandwidth or latency becomes anyone's problem [4]. What sits behind that fog tier in this design is still recognisably a cloud service in NIST's sense of the term (provisioned on demand, billed only for what actually runs, and elastic enough to absorb a burst without anyone pre-sizing a server for the worst case [5]) rather than a second fixed installation mirroring the fog node's own constraints. This project puts that two-tier split to work on a scaled-down but structurally faithful pair of bridge spans, designated span-a and span-b, each carrying the same five sensor types running as independent containers.

What actually happens to a reading, concretely: it arrives at the fog node over HTTP and sits in a lock-protected buffer until the next window boundary, at which point every reading sharing a sensor type and span is reduced to one

min/max/average/latest summary and checked against four threshold rules before anything is sent onward; the raw samples themselves never leave the fog host. From there a cloud-side pipeline takes over persistence, and the dashboard is where those persisted summaries are finally put back together: a composite integrity score per span, five raw per-sensor reading rows underneath it, and a rolling strain trend chart tracking the signal the index weighs most heavily. This report treats four claims as things to be shown running rather than asserted: that a fog layer here actually reduces traffic through real windowing and threshold logic rather than relaying every sample unmodified; that the AWS services standing in for a backend absorb bursty, on-demand load the way a permanently-provisioned process could not; that folding five raw numbers into one integrity score per span is a defensible simplification rather than a loss of information; and that the entire stack runs, not merely compiles, against a real AWS account separate from the LocalStack emulation most of development iterates against.

The remainder of this report follows that structure. Section II sets out the architecture and the reasoning behind its component choices, including the integrity index formula and the system's security posture. Section III covers the software implementation, the automated test suite, continuous integration, and the real AWS deployment together with the defects a pre-deployment audit found and corrected. Section IV evaluates the system against both the test suite and live evidence gathered from the deployed endpoint. What Section V adds at the end is a critical assessment of the design's trade-offs, its limitations, and where the work could go next.

II. System Architecture and Design

Fig. 1 shows the deployed system's six moving pieces: the sensor containers and fog node sharing one EC2 instance, an SQS queue, two independently deployed Lambda functions, a DynamoDB table, and a browser-facing S3 bucket reached through API Gateway.

A. Sensor and Fog Layer

Each of the ten sensor containers (five sensor types across two spans) runs two independent operating-system processes rather than two threads or a single asynchronous loop, coordinated only by a multiprocessing event flag and sharing no Python-level memory. A sampling process perturbs a bounded random walk on its own fixed interval and places a timestamped reading onto a process-safe queue; a separate dispatch process drains that queue on its own, independently configurable interval and posts the accumulated batch to the fog node, retrying on the next tick rather than dropping a batch if the network call fails. Splitting sampling from dispatch this way means a slow or failing network call in the dispatch process can never stall the sampling process's own timing, which matters for a sensor whose physical read cadence should not depend on network conditions, a concern the wireless bridge sensor literature raises directly when reliable local buffering is absent between the sensing element and the network hop [2]. Every sensor container's sample and dispatch intervals are configured independently of one another; the live deployment, for instance, samples deck vibration once per second while dispatching only every six seconds, batching six samples per network round trip rather than sending each one individually.

The fog node itself is a Bottle application served by a real threaded, multi-connection WSGI server built from the Python standard library rather than Bottle's built-in development server. A background thread wakes once per configured window (ten seconds in the live deployment) and swaps the buffer for an empty one under a lock, so ingestion and flushing never contend for the same list at the same instant. The swapped-out readings are grouped by sensor type and span, reduced to a count/min/max/average/latest summary per group, and checked against four threshold rules (an average-strain rule, a maximum-vibration rule, an average-tilt rule, and an average-traffic-load rule), each expressed as a small immutable record dispatched through Python's structural pattern matching rather than a dictionary of comparison callables. The fifth sensor type, expansion-joint movement, carries no rule at all: it is deliberately informational, tracking thermal contraction and expansion (and therefore permitted to go negative, unlike every other reading) without being folded into an alert decision.

B. Backend: Queue, Function, and Store

The fog node never calls the ingestion Lambda directly. Instead, every window's finished summaries are placed on an SQS queue, which absorbs bursts as backlog rather than as failed calls when the ingestion function's available concurrency is temporarily exceeded, and which decouples the fog node's own request rate from whatever cold-start or throttling behaviour the Lambda side is experiencing at a given moment. That decoupling comes at the cost of the weaker consistency guarantees an asynchronous queue implies compared to a direct synchronous write, a trade-off Vogels characterizes generally for large-scale distributed storage and messaging, where availability and partition tolerance are bought by relaxing how quickly every reader is guaranteed to observe every write [6]. A window's data is visible to the dashboard only after it has cleared the queue and been written by the ingestion Lambda, not the instant the fog node flushes it. Messages are placed on the queue in batches (an entire window's worth of summaries chunked at the ten-entry limit the SQS batch API imposes) rather than one call per summary, which keeps the number of publish calls proportional to the number of distinct (sensor type, span) groups that closed in a window rather than to how many individual readings arrived.

Both Lambda functions are invoked on demand rather than run continuously: the ingestion function fires once per SQS batch delivered by its event source mapping, and the dashboard function fires once per API Gateway request. Neither is billed while idle, a property serverless computing surveys identify as the central economic argument for a FaaS-shaped backend over a permanently provisioned server whenever a workload's traffic is naturally bursty rather than steady [7], which holds here by construction, since a window only produces a message

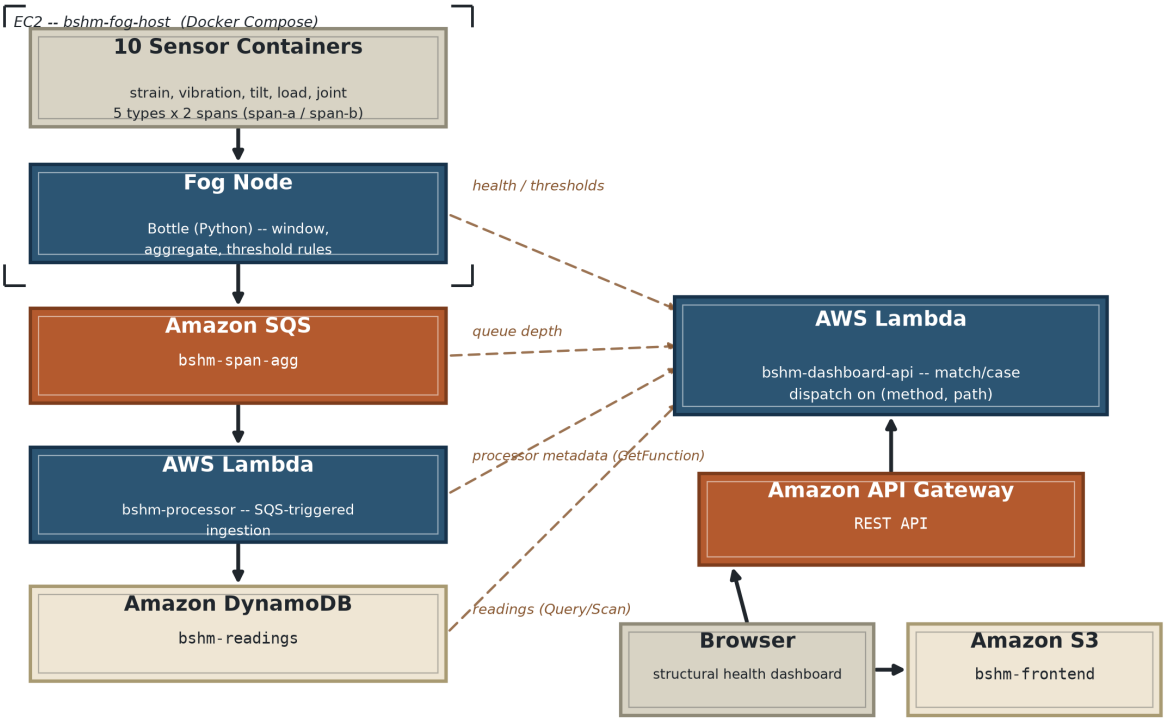

Fig. 1. Deployment topology. Ten sensor containers on a single EC2 instance dispatch to the co-located fog node, which batches window aggregates onto SQS. Lambda bshm-processor consumes the queue into DynamoDB; Lambda bshm-dashboard-api, invoked through API Gateway, answers the browser and pulls its own four read dependencies (dashed edges): fog health/thresholds, SQS queue depth, bshm-processor's own metadata, and DynamoDB itself.

when a sensor group actually reported something during it. DynamoDB backs the store because its partition-key-driven routing supplies the dashboard's dominant query shape, "the most recent N windows for this sensor type," as a single partition query rather than a table scan; the table's partition key is the sensor type itself, and its sort key concatenates each window's end timestamp with the originating span, since two spans flushing in the same ten-second window would otherwise collide on a bare timestamp key.

The dashboard's own API, answering behind API Gateway REST API pe87xzlj3j, is a distinct Lambda from the ingestion function. It dispatches on the incoming HTTP method and path through a single Python structural-pattern-matching statement over a five-way tuple of route patterns (the pattern-matching specification describes exactly this style of multi-way dispatch on a tuple's shape as one of its intended uses [8]), delegating every route's actual work to the same data-access and thresholds-proxy logic the LocalStack-profile dashboard already used rather than re-implementing that logic behind the Lambda entry point. Every response this function returns, including its 400 and 404 error paths, carries an unconditional cross-origin header, discussed further in Section II-C.

C. Structural Integrity Index and Security Posture

The dashboard's primary per-span figure is not any single raw reading but a composite score folding two of the five sensor types, strain and deck vibration, into one 0–100 index:

$$S_{\text{strain}} = \text{clamp}_0^{100}\left(100 - 100\,\frac{\bar{x}_{\text{strain}} - 400}{1200 - 400}\right) \tag{1}$$

$$S_{\text{vib}} = \text{clamp}_0^{100}\left(100 - 100\,\frac{x_{\text{vib}}^{\max} - 8}{20 - 8}\right) \tag{2}$$

$$I = \text{round}\left(\frac{S_{\text{strain}} + S_{\text{vib}}}{2},\, 1\right) \tag{3}$$

where $\bar{x}_{\text{strain}}$ is a window's strain average in microstrain and $x_{\text{vib}}^{\max}$ is a window's peak deck vibration in millimetres per second. Each component score is 100 at or below a safe bound (400 microstrain, 8 mm/s) and 0 at or beyond a critical bound, interpolating linearly between them. The critical bounds are not independently chosen figures: they are numerically identical to the fog node's own alert thresholds (1200 microstrain average, 20 mm/s peak vibration), so the index is constructed to reach exactly zero at the same point an engineer reading the dashboard would already see an active alert badge, rather than converging toward zero asymptotically past that point. The resulting index is banded into five qualitative labels

(excellent, good, fair, poor, critical) at fixed floors for the dashboard's headline display. Tilt, traffic load, and expansion-joint movement are deliberately excluded from the index itself, a scope decision this report returns to in Section V.

Security posture follows the principle of least exposed surface rather than an added authentication layer, appropriate to a demonstrator whose data is not sensitive. The EC2 instance hosting the fog node and sensors exposes only inbound TCP port 8000 through its security group, with no SSH access configured; all instance administration takes place through Systems Manager rather than a key pair. Neither AWS-facing Python module in the deployed code constructs explicit static credentials anywhere: both the fog node's SQS client and the dashboard Lambda's DynamoDB, SQS, and Lambda clients rely on boto3's default credential-provider chain, which resolves to the EC2 instance profile on the fog host and to the Lambda execution role inside both functions automatically. Because the dashboard frontend is served from an Amazon S3 bucket while its API answers from a separate API Gateway domain, every cross-origin browser request from that frontend is subject to the same-origin policy the Web Origin Concept formalizes [9]; the dashboard Lambda answers this by attaching an unconditional wildcard cross-origin header to every response, including error responses, rather than only to successful ones, so a client-side failure path is never silently blocked by the browser without a visible network error.

III. Implementation

A. Software Stack

The entire system is Python throughout. The fog node and the LocalStack-profile dashboard each independently choose Bottle as their HTTP micro-framework, a coincidence of two separate design decisions within the same project rather than a shared convention, since the two components have no code in common beyond the boto3 SDK and standard library. Both are served by the same hand-assembled threaded WSGI server pattern from the standard library rather than any bundled development server, so both the local test harness and the production run exercise identical server behaviour. Sensor simulation uses only the multiprocessing and urllib modules from the standard library; no HTTP client or async framework dependency was pulled in for a component whose network traffic is a single POST per dispatch cycle. The dashboard's Chart.js strain-trend chart is vendored directly into the static asset directory rather than loaded from a content delivery network, so the dashboard renders identically whether or not the browser can reach a third-party host.

B. Automated Test Suite

All 115 tests across the fourteen files in Table I pass. Most exercise pure functions or a Bottle route handler in process, but the fog HTTP tests take a materially different approach for eight of their ten cases: rather than calling the fog application through Bottle's in-process test client, which never opens a network socket, those eight bind the exact threaded WSGI server class the production entry point uses to an ephemeral localhost port, start it on a background thread, and drive it with real urllib HTTP requests over an actual TCP connection. That distinction matters for at least one case in the file directly: a request body that is not valid JSON at all (rather than a well-formed JSON document missing a required field) has to be rejected by whichever layer actually parses the request body first, and only a real request against a real running server exercises that parsing path the way a live sensor's malformed payload would. The remaining two cases in the same file skip the socket layer entirely and call the fog application's flush routine directly, verifying through a queue-client test double that records every SQS call it receives that a single flush window closing two separate sensor groups issues exactly one batched publish call carrying both messages rather than two individual calls, the batching behaviour introduced during the pre-deployment audit described in Section III-C.

TABLE I
Automated Test Suite by Area

Test Area	Tests
Sensor simulation	8
Fog ingest buffer	7
Fog window math	4
Fog threshold rules	9
SQS batch publishing	5
Fog ingest payload checks	12
Fog HTTP (real socket)	10
Message-to-item transform	6
Ingestion entry point	3
Integrity index	10
DynamoDB query and pagination	13
Fog-to-dashboard proxy	2
Dashboard routes	13
API Gateway dispatch	13
Total	**115**

C. Continuous Integration

A GitHub Actions workflow runs on every push and pull request against the repository, split into two dependent jobs. The first installs Python 3.12 and the project's pinned test dependencies and runs the full pytest suite; the second only starts once the first has succeeded, brings up the complete LocalStack-backed Docker Compose stack, and runs a separate end-to-end verification script against the live containers to confirm a reading actually reaches DynamoDB through the whole pipeline rather than only checking that individual functions return correct values in isolation. Container logs are captured automatically on failure, and the stack is torn down unconditionally afterward so no LocalStack containers linger between workflow runs.

D. AWS Deployment and Defects Found

Beyond the LocalStack profile the CI workflow and local development both run against, the system was deployed to a real AWS Academy Learner Lab account (661886400169), provisioned under this project's own student login and confirmed via the account identity returned by the AWS CLI before any resource was created. Provisioning went through

a reusable Terraform module, applied once to create 24 resources with no manual AWS CLI steps, an infrastructure-as-code approach whose central claimed benefit over a sequence of manual console or CLI actions is that a system's resource topology becomes a versioned, reviewable artifact in its own right [10]. The module's local state file, left over from a previous unrelated deployment through the same module, already tracked a different set of live resources under the default Terraform workspace; applying this project's variables directly against that workspace would have made Terraform plan to destroy those resources rather than create this project's alongside them, since the state and the new variable file would then disagree about what ought to exist. A dedicated workspace was created and switched into first, isolating this project's state from that prior deployment, and only then was the apply run.

A code audit carried out before the deployment, rather than after, found and corrected two defects. The first was a DynamoDB scan-pagination undercount in the item-count helper used by the dashboard's backend-stats endpoint: the original implementation issued one count-only scan call and returned its count directly, correct only while a table's scanned data stays under the roughly one-megabyte limit a single scan call is subject to. Beyond that limit DynamoDB signals there is more to read through a continuation key in its response, and a caller that never follows it silently reports only the first page's count. The fix is a small class implementing Python's iterator protocol directly, an object that requests one page per call and stops once no continuation key comes back, summed with the built-in sum function, rather than a generator function, recursion, or the AWS SDK's own paginator helper. The second was unbatched SQS publishing: the fog node's flush cycle previously issued one send per closed sensor group, correct but unnecessarily costly whenever more than one group closes in the same window, which is the normal case across two spans and five sensor types. The fix chunks a whole window's summaries into batched sends of at most ten entries and is invoked once per flush cycle rather than once per message. No credential-handling defect was found in this project's code, since every AWS-facing module already relied on boto3's default credential-provider chain without ever constructing an explicit static credentials object.

Verification against the live deployment took place within roughly a minute of the EC2 instance completing its container bootstrap. The health endpoint reported all four pipeline fields (gateway reachability, queue reachability, ingestion Lambda activity, and overall pipeline freshness) true, with the freshest window's age under ten seconds. The backend-stats endpoint reported one hundred items already landed in DynamoDB, and the spans endpoint returned a real, climbing structural-integrity-index history for both spans rather than a fixture: one observed sample showed span-a's index moving from 100.0 down to 91.2 and then 84.7 across its first three windows as real generated strain and vibration readings arrived. All four static frontend assets served from the S3 bucket returned success directly, uploaded with the correct nested static-asset path structure from the first deploy, avoiding the broken-asset-reference failure mode a flat, unprefixed bucket sync would otherwise produce.

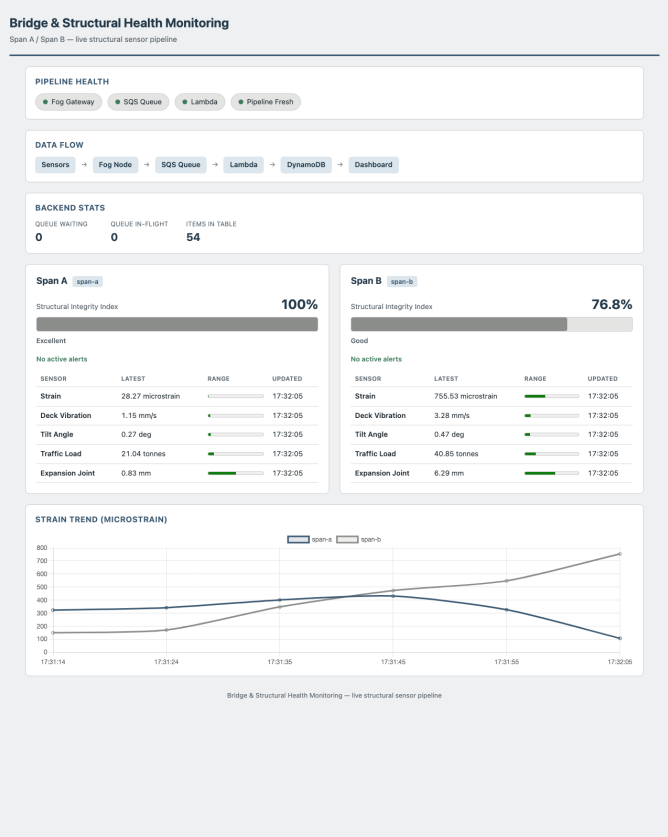

Fig. 2. The deployed dashboard captured mid-session: pipeline health indicators, backend queue and table statistics, both spans' structural integrity index and per-sensor reading rows, and the rolling strain trend chart.

IV. Evaluation

Fig. 2 shows the deployed dashboard rendering live data pulled through the API Gateway endpoint. Span-a's integrity index sits at 100% (excellent band, no active alerts) while span-b's sits lower on the same scale, both driven directly by the strain and vibration windows arriving through the pipeline rather than by fixture data — the visible gap between the two spans' index values is itself evidence that the scoring formula is sensitive to the actual difference in each span's simulated readings rather than saturating at either extreme regardless of input.

Table I's 115 passing tests give the strongest coverage to exactly the components with the most decision logic: 23 tests across the data-access and scoring suites cover DynamoDB query shaping, pagination, and the integrity-index formula's clamping behaviour at and beyond both safe and critical bounds; 21 across the alert and validation suites cover threshold evaluation and malformed-payload rejection; and the two HTTP-route suites together (fog and dashboard) commit 23 tests to routing and status-code behaviour, 21 of them against a genuinely live socket rather than an in-process

client. That suite is, by construction, incapable of validating certain classes of defect that only a running deployment can surface: whether API Gateway's own request and response transformation preserves a header exactly as the Lambda code set it, whether an IAM role attached to a Lambda or an EC2 instance profile actually grants the permissions the code assumes, and whether the EC2 security group's port restriction still permits every path the fog node itself depends on. Those were checked only by exercising the live deployment directly rather than inferred from the local suite, and the pagination and batching fixes from Section III-C both received dedicated regression tests alongside their manual verification rather than being confirmed by inspection alone: a four-page synthetic scan of unequal sizes asserts the item-count helper sums every page correctly rather than only the first, and a twenty-three-message window asserts the publisher issues exactly three batch calls of ten, ten, and three entries rather than twenty-three individual calls.

One property the deployed evidence demonstrates that the LocalStack-backed test suite cannot is end-to-end latency through a real network path spanning an EC2-hosted fog node, a managed queue, two independently cold-starting Lambda functions, and a managed key-value store, rather than every component running as sibling processes on one machine. The freshest-window age reported under ten seconds against a ten-second flush window indicates that path adds only a small fraction of the window interval itself, consistent with the fog node's flush cycle, rather than the queue-to-Lambda-to-store chain, being the dominant contributor to end-to-end delay.

V. Critical Analysis and Conclusion

Several design choices in this system trade one property for another rather than being unambiguously correct, and are worth stating plainly rather than leaving implicit. Placing all buffering and windowing logic behind a single in-process lock, rather than partitioning the buffer by sensor type or span, keeps the fog node's ingest path simple at the cost of every concurrent request briefly contending for the same lock; at the ten-sensor scale this project runs at, that contention is negligible, but it would not necessarily remain so at the scale of an actual multi-span bridge instrumented with hundreds of individual gauges. Batching SQS publishes at the ten-entry API limit reduces call volume but means a single malformed message inside an otherwise-valid batch can, depending on how batch partial failures are handled downstream, affect the delivery of messages it was never actually related to, a risk the current implementation accepts without partial-failure handling of its own, since no batch has yet failed partially against real traffic.

The structural integrity index itself is a deliberate simplification rather than a comprehensive damage indicator. Folding only strain and vibration into the score, while leaving tilt, traffic load, and expansion-joint movement as separately displayed raw readings, was chosen because strain and vibration are the two structural signals with the clearest, most directly interpretable relationship to load-bearing capacity in the wireless-sensor bridge monitoring literature [11]; a bridge experiencing a genuine deformation event captured by rising tilt readings, without a corresponding rise in strain or vibration, would not be reflected in the index at all under the current formula. Extending the index to weight all four alert-bearing sensor types, rather than only the two currently folded in, is a natural next step this project defers rather than one it argues against on principle. Similarly, the dashboard's history-fetching logic over-fetches by a fixed multiplier to reliably reconstruct one span's recent window sequence from a table interleaving two spans' rows by timestamp; that multiplier is calibrated for a two-span bridge and is not guaranteed to remain sufficient without adjustment if the number of monitored spans grew substantially.

What this report's evidence cannot claim is worth being explicit about. The 115-test suite runs against LocalStack and in-process doubles, and while that setup exercises the pagination, batching, and threshold logic in real depth, it has no way of touching what only a live API Gateway invocation can show: whether the IAM permissions attached to the EC2 instance profile and each Lambda's execution role are as wide as the code assumes, whether the network path between the fog host and the managed services it calls behaves the way a same-machine LocalStack call always will, and whether a header the Lambda sets actually survives API Gateway's own request and response mapping. Section III-D's live-deployment pass is the only place any of that was actually checked. A second gap sits alongside it: everything gathered from the live account so far comes from a single observation window taken shortly after deployment finished, not a sustained run under load. A load-testing script already exists and has been run against the LocalStack profile, but nobody has yet pointed it at the real SQS queue and both live Lambda functions long enough to find this particular account's actual concurrency ceiling.

Beyond folding all four alert-bearing sensor types into the integrity index, two further extensions would be worth pursuing. Smart, self-diagnostic sensor nodes capable of flagging their own degraded readings before those readings ever reach the fog node's aggregation logic are an active area the broader smart-sensing literature for structural monitoring surveys directly [12], and would let this system distinguish an actual structural event from a failing sensor rather than aggregating both identically. And a sustained load test against the live account, exercising the same batching and windowing logic this report evaluates against a fixed set of unit tests, would convert the single-observation-window evidence gathered here into a measured throughput and latency ceiling under conditions closer to continuous operation than a short verification pass can establish.

The Terraform workspace near-miss described in Section III-C is, in hindsight, the moment that shaped how the rest of this deployment was checked: a state conflict that Terraform's own plan output flagged before anything was applied, not something the tooling caught after the fact, and treating that warning as worth reading carefully rather than clicking past set the tone for everything that followed. The same instinct,

verifying a code-level fix against the actual running system rather than assuming it behaves identically once real AWS services and real network latency enter the picture, is what let the pagination and batching fixes be confirmed twice over: once by a dedicated unit test, and again by watching the deployed dashboard's own numbers move correctly. What this report set out to show at the start, that continuous, locally-aggregated sensing across two bridge spans could be reduced to one interpretable figure per span and run on infrastructure that only pays for itself while it is actually doing something, is now backed by more than a design on paper: a fog layer that genuinely windows and alerts, 115 tests exercising that logic and the DynamoDB/SQS plumbing beneath it, and a live AWS account, 661886400169, whose pagination and batching defects were caught and fixed before any real traffic ever reached them. The full implementation, including every module and test file referenced in this report, is available at [GitHub repository URL].

REFERENCES

[1] C. R. Farrar and K. Worden, "An introduction to structural health monitoring," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 365, no. 1851, pp. 303–315, 2007.

[2] S. Kim, S. Pakzad, D. Culler, J. Demmel, G. Fenves, S. Glaser, and M. Turon, "Health monitoring of civil infrastructures using wireless sensor networks," in *Proc. 6th Int. Conf. Information Processing in Sensor Networks (IPSN '07)*, Cambridge, MA, USA, 2007, pp. 254–263.

[3] J. M. Ko and Y. Q. Ni, "Technology developments in structural health monitoring of large-scale bridges," *Engineering Structures*, vol. 27, no. 12, pp. 1715–1725, 2005.

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12)*, Helsinki, Finland, 2012, pp. 13–16.

[5] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-145, 2011.

[6] W. Vogels, "Eventually consistent," *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, 2009.

[7] I. Baldini, P. Castro, K. Chang, P. Cheng, S. Fink, V. Ishakian, N. Mitchell, V. Muthusamy, R. Rabbah, A. Slominski, and P. Suter, "Serverless computing: Current trends and open problems," in *Research Advances in Cloud Computing*, S. Chaudhary, G. Somani, and R. Buyya, Eds. Singapore: Springer, 2017, pp. 1–20.

[8] B. Bucher and G. van Rossum, "PEP 634 – Structural Pattern Matching: Specification," Python Enhancement Proposals, Python Software Foundation, 2020, accepted for Python 3.10. [Online]. Available: https://peps.python.org/pep-0634/

[9] A. Barth, "The Web Origin Concept," IETF RFC 6454, 2011.

[10] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*, 1st ed. Sebastopol, CA: O'Reilly Media, 2016.

[11] J. P. Lynch and K. J. Loh, "A summary review of wireless sensors and sensor networks for structural health monitoring," *The Shock and Vibration Digest*, vol. 38, no. 2, pp. 91–128, 2006.

[12] B. F. Spencer, M. E. Ruiz-Sandoval, and N. Kurata, "Smart sensing technology: Opportunities and challenges," *Structural Control and Health Monitoring*, vol. 11, no. 4, pp. 349–368, 2004.